

Operating Systems

Lab experience - shared memory and pipes

Mirco Marchetti, Riccardo Lancellotti

SECloud research group
University of Modena and Reggio Emilia

AY 2025/26

Goals of this lab experience:

- Understand and use shared memory for inter-process communication
- Understand and use pipes for inter-process communication

Shared memory

- Review the source code of
`02-anonymous-producer-consumer.c`
`03-shm-producer.c` and
`03-shm-consumer.c`
- Compile and run them, and check that they work as expected

Exercise 1

- Write a C program that creates an anonymous shared memory region and then invokes `fork()` to generate a child process. Immediately after being created, the child process must suspend by calling `pause()`, waiting for a signal from the parent.
- The parent process repeatedly prompts the user to enter a line of text from standard input. Each line must be stored in the shared memory region. The parent notifies the child by sending `SIGUSR1`, and then suspends itself by calling `pause()` until the child completes its work.
- Upon receiving `SIGUSR1`, the child process reads the string stored in shared memory, converts all alphabetic characters to uppercase, and prints the resulting string to standard output. Once the processing is complete, the child signals the parent with `SIGUSR2` and then waits again by calling `pause()`.
- After receiving `SIGINT`, the parent must ensure that both parent and child terminate gracefully.

Exercise 1: hints

- Start from `exercise-1-template.c`, which you can find in Moodle, within the `Laboratorio shm pipe` folder
- You can use the `mmap` system calls to create and manage the shared memory region
- You can use the `kill` system call to send signals to the child process
- The child can use `pause()` to wait for signals, and the parent can also use `pause()` to wait for signals from the child

Exercise 2

- Extend the previous program by turning the shared memory region into a bidirectional communication area. In addition to the input string written by the parent, the shared memory must also contain a second buffer for the processed output and a status flag describing the current state of the exchange.
- At each iteration, the parent reads a line of text from standard input, stores it in the input buffer of the shared memory region, updates the status to indicate that new data is available, and sends SIGUSR1 to the child.
- Upon receiving SIGUSR1, the child reads the input string from shared memory, converts it to uppercase, writes the transformed string into the output buffer, updates the status to indicate that processing is complete, and sends SIGUSR2 to the parent.
- ...

Exercise 2 (Continued)

- The parent, after receiving SIGUSR2, reads the processed string directly from the output buffer in shared memory and displays it. The interaction then continues with the next user input.
- The program must still terminate gracefully when the parent receives SIGINT, also ensuring that the child is terminated correctly.

- The shared memory can be used to store a structure similar to the following one, which contains an input buffer, an output buffer and a status flag. The status flag can be used by the parent and the child to coordinate the exchange of data and ensure that they are not accessing the shared memory at the same time.

```
#define SHM_SIZE 1024
#define EMPTY 0
#define DATA_READY 1
#define RESULT_READY 2

typedef struct {
    char input[SHM_SIZE];
    char output[SHM_SIZE];
    int status;
} shared_data;
```

Pipes

- Review the source code of
 - `01-pipe.c`
 - `02-pipe-reader-blocked.c`
 - `03-pipe-writer-blocked.c`
- Compile and run them, and check that they work as expected

Exercise 3

- Carefully read and understand the source code of `05-pipe-ring.c` and `06-pipe-star.c`
- Compile and run them, and check that they work as expected
- Make sure to understand which pipes are closed, how and why!

Exercise 4: Guess the number

- Write a C program that creates two pipes and then invokes `fork()` twice to generate two child processes. Each child process must communicate with the parent through its own dedicated pipe.
- Each child process must install a signal handler for `SIGUSR1` and then suspend its execution using `pause()`, waiting for the parent to start a new round.
- The parent process implements a game that proceeds in multiple rounds. At the beginning of each round, the parent waits for the user to press ENTER, then generates a random integer between 1 and 10 (the *secret number*).
- The parent then sends `SIGUSR1` to both children to signal the start of the round.
- ...

Exercise 4: Guess the number (continued)

- Upon receiving SIGUSR1, each child generates a random integer between 1 and 10 (its *guess*), and writes a message to the parent through its pipe. The message must contain both the child's process identifier and the generated guess.
- The parent reads one message from each pipe, retrieves the guesses, and prints the secret number along with both children's guesses.
- The parent determines which guess is closer to the secret number. The child with the closest guess wins the round and receives one point. In case of a tie, both children receive one point.
- The game continues until one of the two children reaches a predefined score (e.g., 5 points). At that point, the parent declares the winner.
- Finally, the parent terminates both children, closes all pipe descriptors, and waits for the child processes to exit before terminating.